

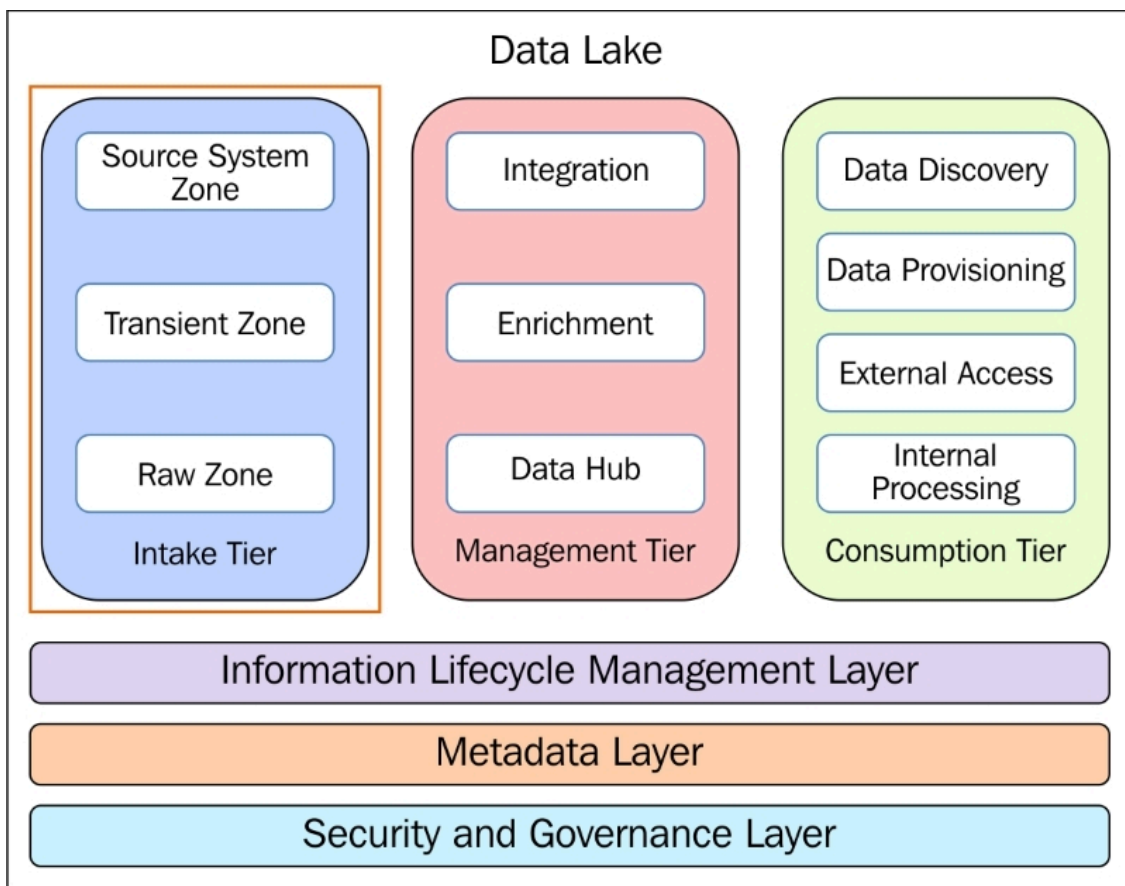
Chapter 2. Data Intake

In the preceding chapter, you understood the need for Data Lake and gained a high-level understanding of the key components that can comprise a Data Lake and how important each of these components are for building it. We have seen how the changing business landscape is provoking data growth and how organizations are adopting a newer paradigm such as Data Lake to ingest this data and extract analytical value.

In this chapter, you will understand in detail, the Intake Tier that was introduced in [Chapter 1](#), *The Need for Data Lake*. The following topics will be covered in this chapter:

- The process of obtaining data into the Data Lake's Intake Tier
- An high-level overview of the various External Data Sources from which the data can be acquired, and the variety of data that can be ingested
- The key functionalities that can be implemented as part of the Data Intake Tier
- The various data intake modes
- Big Data tools and technologies that can be used to acquire a variety of data from external sources; architectural guidance on choosing the tools for Data Intake

The following figure represents the end state architecture of the Data Lake as discussed in [Chapter 1](#), *The Need for Data Lake*. In the subsequent sections, we will be discussing in detail, the highlighted Intake Tier of the Data Lake end-state architecture:



Data Lake end-state architecture

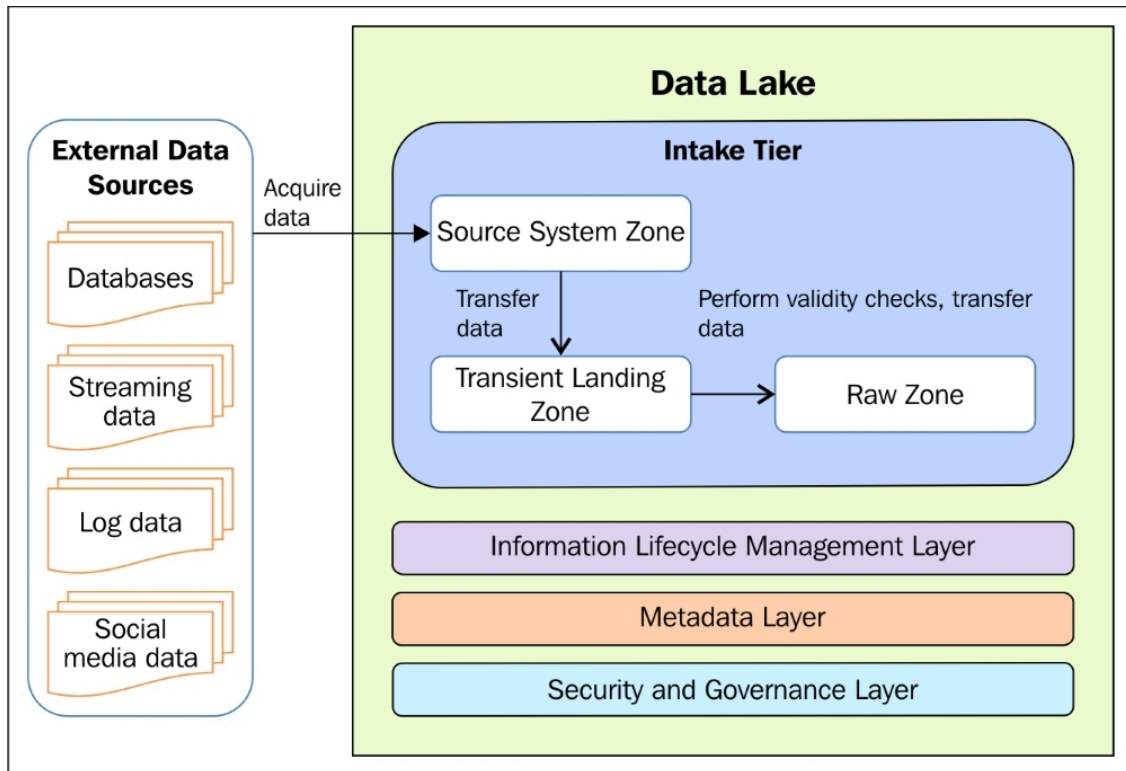
Understanding Intake tier zones

Enterprises sit on vast reserves of diverse, potentially invaluable data such as databases, social media, logs, and sensor data locked in data silos. The Data Lake is schema-less and stores data of any type and format. Not only storing, but offering the ability to integrate data from disparate sources and ingest high-velocity, multi-structured and massive datasets. This key feature empowers enterprises to perform exploratory and advanced data analysis on all the data and quickly gain actionable insights.

Before the Data Lake is utilized to its hilt to perform data analysis, there should be mechanisms implemented to seamlessly connect the Data Lake to various external data sources and acquire data from them. The Intake Tier in the Data Lake architecture implements the functionalities needed to address this. In the following subsection, let us study the Intake Tier in detail.

Let us start by understanding the data flow from the **External Data Sources** to the Intake Tier. The following figure illustrates the data flow

to the Data Intake Tier:



Flow of data in Data Intake Tier

In the preceding image, the data flows in the following order:

1. The **Source System Zone** establishes the connectivity to the **External Data Sources**.
2. It accesses the data, extracts the required data, and transfers the data into the **Transient Landing Zone** for it to perform basic validity checks.
3. After the validity checks have been performed, the data is moved into the **Raw Zone** to make it accessible for further processing.

The following sections help you understand the features of each zone in the Intake Tier.

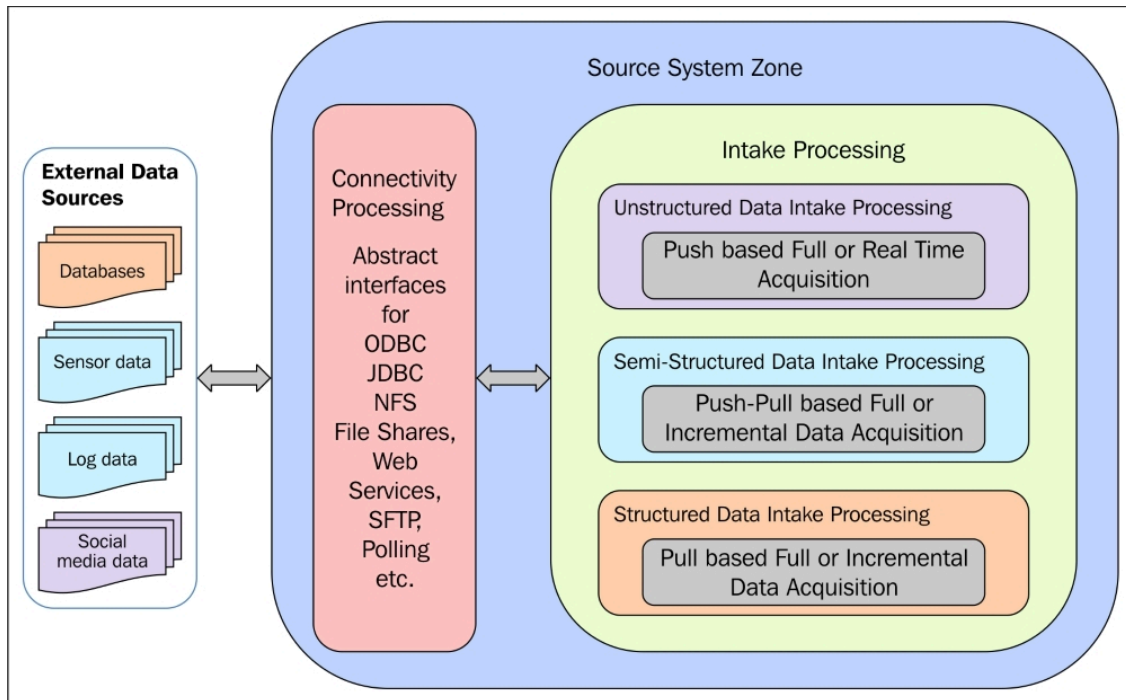
Source System Zone functionalities

This section introduces the Source System Zone and helps you to understand its various features.

The Source System Zone abstracts the connectivity and processing services that are needed to acquire data from the External Data Sources.

This zone provides the interfaces required to implement all the connections to external file shares, databases, and sensor devices. The data can be acquired using various time-based acquisition strategies such as real-time acquisition or batch acquisition, and the data can be loaded incrementally or all at once.

The following image gives you a quick snapshot of the functionalities that can be performed in the Source System Zone:



Source System Zone functionalities

Understanding connectivity processing

The connectivity layer in the Source System Zone implements mechanisms to connect to a variety of external data sources such as databases, social media chatter, logs, and sensor. Each of these data sources differ in the way they have to be accessed, connected, polled, and retrieved, and eventually their respective data is acquired.

Some of the key functionalities and interfaces implemented by this layer are as follows:

- Establishing connectivity to structured databases
- Establishing connectivity to file shares
- Periodically polling for the availability of data from external sources

- Establishing connectivity to web resources such as SFTP and REST APIs
- Establishing connectivity to sensors, devices, and cloud platforms
- Establishing connectivity to social media APIs such as Twitter
- Connection pooling, connection retry, and time-out mechanisms

Once the connectivity layer has established the connectivity with External Data Sources, the **Intake Processing** performs the actual data acquisition.

In the next section, we will understand how this is accomplished by understanding it within the context of data variety.

Understanding Intake Processing for data variety

This section takes you through an overview of the data variety, helps you understand why it is important to integrate a variety of data in the Data Lake, and also highlights the role of the Source System Zone in performing Intake Processing.

The Source System Zone's Intake Processing performs a key role in data acquisition, which includes the processing steps that are needed to do the following:

- Poll external data sources for the availability of data
- Acquire data into the Data Lake whenever it is pushed
- Acquire vast amounts of file-based data in batch mode
- Acquire data incrementally (through change data capture)
- Queue real-time streaming data
- Attempt retries or notify whenever data is not acquired properly
- Scale horizontally whenever the acquired data sizes shoot beyond a limit
- Load balance; recover from disasters so that the data is not lost
- Acknowledge or verify the data transfer through handshake mechanisms

The following subsections help you comprehend the details of how the Source System Zone aids in intake processing of various types of data.

Structured data

Structured data is neatly arranged by delimiters so that a machine can understand its structure; it has entities that have a well-defined format and follow a predefined schema. The data is defined by a set of attributes or fields with their data type specifying the type of data each attribute can hold. A few examples of structured data are as follows:

- Data from Online Transaction Processing systems
- Point-of-sale data
- Mainframe data
- Enterprise Resource Planning data
- Data in the form of flat delimited files

The need for integrating Structured Data in the Data Lake

Ingesting structured data from varied sources into the Data Lake enables the integration of data from proliferating databases, transactional systems such as ERP and CRM systems, and legacy systems such as mainframes. This integration eliminates data silos; all of the data on the Data Lake can now be utilized to get a coherent view that can power a new generation of data-aware applications.

Structured data loading approaches

The Source System Zone typically *Pulls* structured data from the External Data Sources by connecting to the data sources, polling it at regular intervals to check for available data, and then initiating the process of actually acquiring the data. All these functionalities are abstracted in the Source System Zone and are implemented specific to the External Data Source.

The process of acquiring the data can be either a full data load or an incremental one as follows:

- **Full Data Load:** The Source System Zone performs full data load when the data is acquired for the first time. The data from the External Data Source is acquired completely so that the data in the Data Intake Tier reflects the snapshot of the data that is available in the External Data Source.

- **Incremental Data Load:** Incremental data load is performed by the Source System Zone to load the changes in the external data source that have occurred since the last successful data import. Here are a few strategies to implement Incremental Data Load:
 - **Time stamps:** Time stamps stored in the database tables can be used to identify new or modified data.
 - **Partitioning:** Range partitioning can be performed on the source table to partition it along a partitioning key such as date; this allows easy identification of the new data.
 - **Change tables:** Change Data Capture (CDC) can be enabled in databases to populate change tables that maintain a record of all the changes. These changes can be extracted from the source system.
 - **Triggers:** Triggers can be used to watch the recently updated records and to update the last modified time stamp value with the current time. Thus, the time stamp column always holds the latest time and date when a given record was last modified.

Semi-structured data

Semi-structured data is just like structured data, but it does not respect the structural constraints at all times. Most of the time semi-structured data is loosely coupled to the enforced schema, and uses the schema as a guide to the structure of the data. The schema in the semi-structured data varies from record to record. Typically, semi-structured data is used to represent complex relationships using a nested approach; this nesting cannot usually be intuitively depicted using relational databases. A few examples of semi-structured data are as follows:

- Data stored or exchanged in JSON, RDF, or XML format
- Application-generated log data
- Clickstream data

The need for integrating semi-structured data in the Data Lake

The ingestion of semi-structured data such as application and web logs into the Data Lake and integrating them with machine data or network logs can help identify hidden patterns from it. These hidden patterns can

be analyzed to predict machine failures and gain Operational Intelligence (OI).

Semi-structured data loading approaches

The Source System Zone acquires semi-structured data either by *Pulling* data from the External Data Sources or waiting until the data is *Pushed* into this zone.

In both cases of Pull and Push, the Source System Zone connects to the External Data Sources and initiates the process of actually acquiring the data. The major difference between Pull and Push is that; with Push, the External Data Sources *Push* the data to the Source System Zone whenever the data is available. Whereas, while *Pulling* the data, the Source System Zone periodically polls the External Data Source for data availability before acquiring the data.

In either cases of Pull or Push, the process of acquiring the data can be either a full data load or an incremental one, as described in the preceding subsection.

Unstructured data

Unstructured data does not conform to any particular predefined internal structure or data model. It is organized in a haphazard way that makes it difficult to parse by traditional approaches. A few examples are as follows:

- Unstructured data in a textual form such as documents, e-mails, and social chatter
- Machine-generated data such as RFID data, sensor data, and so on
- Binary data sources such as medical images, Youtube videos, or audio files

The need for integrating Unstructured data in the Data Lake

The unstructured data that is ingested from many sources, such as social media chatter and clickstreams, can be integrated with historical data on the Data Lake and provisioned to downstream analytical systems. By ap-

plying real-time analytics to this data, organizations can gain significant insights that aid in real-time decision making.

Unstructured data loading approaches

The Source System Zone typically loads unstructured data in continuous streams or a batch depending on the type of data.

Data such as machine logs, sensors, and so on, are *Pushed* into the Source System Zone whenever they are generated.

Unstructured data such as documents, images, and videos, are generally loaded into the Data Intake Tier all at once, just like the full load. These specific data types are generally *Pulled* or *Pushed* into the Source System Zone.

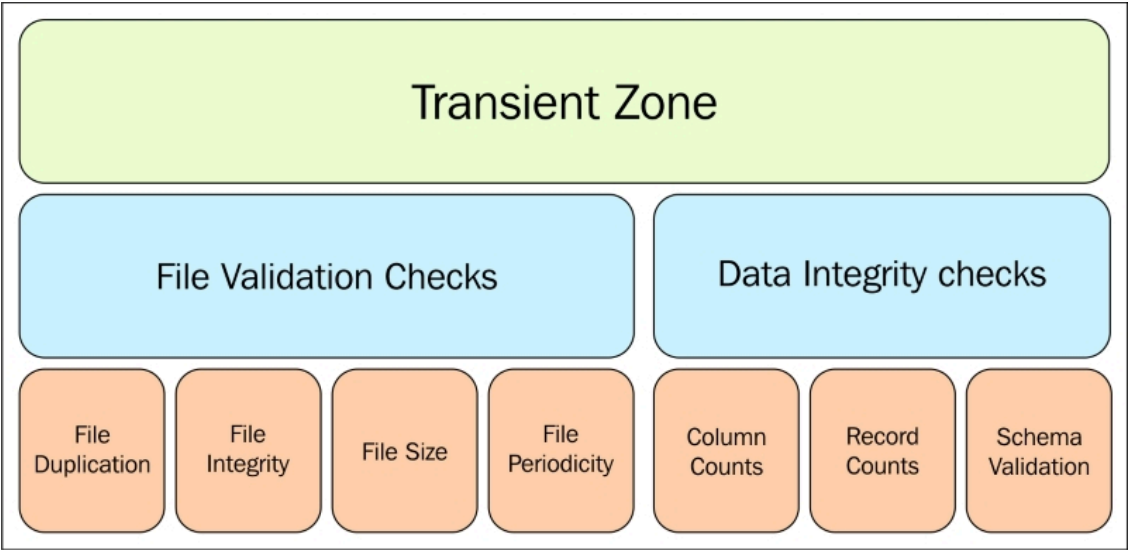
The Source System Zone has connection-processing mechanisms that connect to external data sources, that listen/wait for these logs/events/files, queues them, and finally acquires the data.

Transient Landing Zone functionalities

This section introduces and helps you understand various processing steps such as validations and integrity checks that are performed on the data while it is residing in the Transient Landing Zone.

The Transient Landing Zone is a predefined, secure intermediate location where the data acquired from various External Data Sources will be stored before moving it into the Raw Zone. In the case of batch processing, all the events or data arriving at a periodic interval can be stored into a file. For real-time processing scenarios, the events are streamed into the system, validated in the Transient Landing Zone, and consumed by the subsequent tiers to execute processing on it. Generally, the Transient Landing Zone is a file system-based storage where the data is organized by External Data Sources. Record counts and file-size checks are carried out on the data in this zone before it is moved into the Raw Zone.

The following image gives you a quick snapshot of the steps that could be performed in the Transient Landing Zone:



Transient Landing Zone functionalities

File validation checks

File validation checks are performed to ensure that duplicate files are not ingested, file integrity is maintained, and to check that files are being re-received at periodic intervals. In this subsection, let us go through each of the processing steps that comprise file validation checks.

File duplication checks

File duplication checks are carried out at the Transient Landing Zone in the Intake Tier to ensure that duplicate files are not ingested into the Data Lake. File duplication occurs in cases where incremental data is loaded at periodic intervals; the same data is received in the Transient Landing Zone that was ingested in the previous intake cycle from the same external source.

The deduplication process checks the incoming files for duplicates; it is a three step process listed as follows:

1. **Compare file names:** The file name of the incoming file is compared against the file name of the file that already exists in the Data Lake; if the name is the same, then it is flagged as a duplicate suspect and the next check is performed on it.

2. **Compare the schema:** The schema of the duplicate suspect incoming file is validated with the schema of the file with the same file name that is already stored in the Data Lake. If the schema is the same, then the file is still a duplicate suspect and the next step of checking the content is performed. If the schemas do not match, the file is no more a duplicate suspect.
3. **Compare the content:** This step finds out whether the incoming duplicate suspect file is really a duplicate by comparing the checksum of the incoming file with the file that is already stored in the Data Lake and that has the same file name and schema as that of the incoming file. If the MD5 checksum of the files match, then the incoming file is flagged as a duplicate and the file is deleted. The MD5 checksum of the file is stored as part of the metadata to enable deduplication.

File integrity checks

File integrity checks are performed to verify that the file has been received at the Transient Landing Zone without any errors in transmission; the integrity is verified by using message digests. The message digest of the file that is being transferred can be calculated at the External Data Source and sent as part of the metadata when the file is pushed for ingestion. After the data intake is complete, the message digest of the received file is calculated in the Transient Landing Zone and is compared with the value sent by the External Data Source. If the values do not match, the file integrity is lost and the appropriate actions, such as reinitiation of the intake or sending notifications for manual intervention can be performed.

File size checks

The file size of the file that is being transferred can be calculated at the external data source; this file size would be part of the metadata when the file is pushed for ingestion. The file size on the external source can be validated at the Transient Landing Zone after the data intake is complete at the Data Lake. If the file size does not match, the files have to be flagged appropriately and the next course of action, such as reinitiation of the intake or sending notifications for a manual intervention can be performed.

File periodicity checks

File periodicity checks ensure files are received at periodic intervals: Automated checks can be put in place to monitor directories where the files would be ingested at periodic intervals. If the files are not being ingested, this process would send alerts or notifications informing that files are not being ingested.

Data Integrity checks

Data Integrity checks are carried out on the data at the time of data intake in order to ensure that all the data that was ingested from an External Data Source has been received at the Transient Landing Zone successfully without losing its integrity. The following sections discuss a few techniques used for Data Integrity checking.

Checking record counts

Validating record counts ensures Data Integrity and enables identification of missing records, if any. While acquiring data from External Data Sources such as databases, the record counts can be calculated at the external source and captured as part of the metadata. In the case of structured delimited files, the record counts on the external data source will be part of the metadata while the file is ingested. These record counts are matched at the Transient Landing Zone after the Data Intake is complete. If the record counts do not match, the respective files are flagged appropriately and the next course of action such as reinitiation of the intake or sending notifications for a manual intervention can be performed.

Checking for column counts

The number of columns that are part of data extraction can be captured and stored as part of the metadata. The column count on the external source is validated with the column count on the Transient Landing Zone after the intake process is complete. If the column counts do not match, the files are flagged appropriately and the next course of action such as reinitiation of the intake or sending notifications for a manual intervention can be performed.

Schema validation checks

Schema validation checks are performed on the data in the Transient Landing Zone to validate if there are any structural changes in the schema of the External Data Source since the last data ingestion.

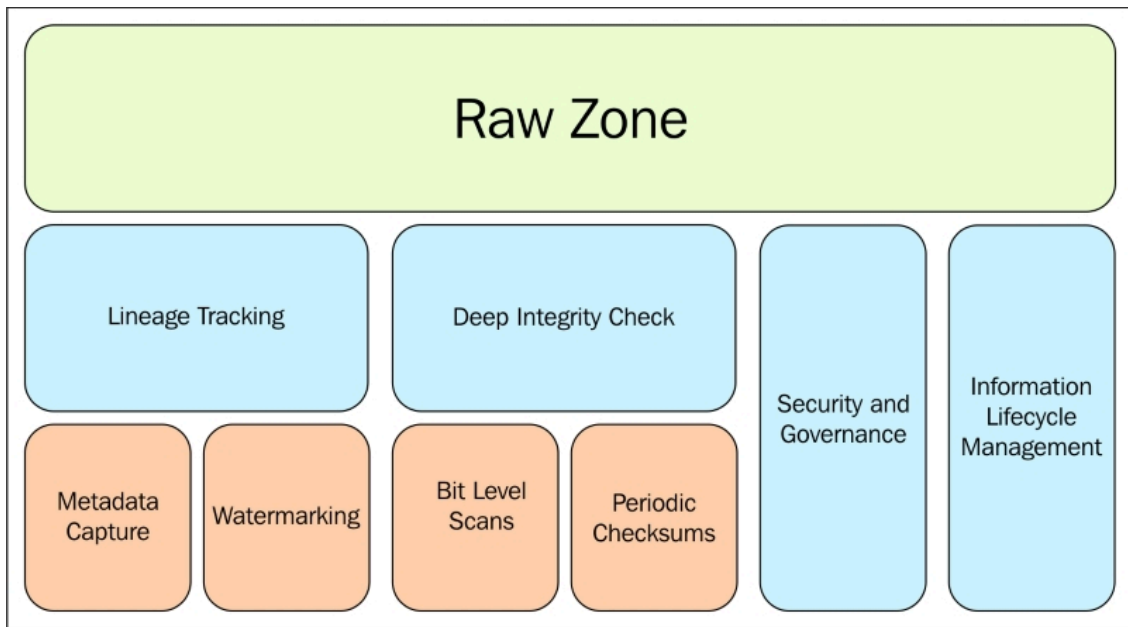
The structure of the data in the Transient Landing Zone is changed whenever there are any structural changes detected in the External Data Source. This ensures that the new data that gets ingested has the same structure as it is in the external source. Avro or Thrift can be used for schema evolution.

Raw Storage Zone functionalities

This section introduces and helps you understand various functionalities of the Raw Storage Zone. These functionalities help capture deep metadata, prevent degradation of data when it is stored for a longer duration, by performing deep bit-level integrity checks and periodic checksums, and enable security and governance mechanisms on the data while it is residing in the Raw Storage Zone.

The Raw Storage Zone is a part of the Data Intake Tier. After validating the incoming records for duplication, schema changes, and integrity in the Transient Landing Zone, the data is ingested into the Raw Storage Zone. This zone contains raw data in its original form, with the original fidelity of the data being preserved. This data is retained for active use and archiving, and will be made available for discovery and provisioning, making it accessible to any downstream systems that require the raw data for performing analytical processing. Deep validation of data is performed here and this data is watermarked for lineage and tracking.

The following image gives you a quick snapshot of the steps that could be performed in the Raw Storage Zone:



Raw Storage Zone functionalities

Data lineage processes

The following subsections help you in understanding the processes that are performed on the data in the Raw Zone to capture data lineage and enable **Lineage Tracking**.

Watermarking process

Watermarking is a specific process performed on structured data to track its lineage. It is applied on the data in the Raw Zone where a unique identifier is appended to each record. It enables to capture changes that were made to the record using this unique identifier. This helps in tracking the complete lifecycle of the data as it moves overtime through various tiers/zones in the Data Lake.

Metadata capture

In this context, Metadata is captured for structured, semi-structured, and unstructured data in the Raw Storage Zone. It collects vital information about the history and provenance of data by capturing information on when the data was ingested into the Data Lake, who initiated the ingestion, and where the data originated from.

The following list gives you a brief overview of the range of information that could be captured as metadata:

- The external data source information.
- The entity information such as table names with row counts, column data type information in case of relational sources or file names, the MD5 checksum of the file, the date and time of ingestion, the destination details, and the retention period.
- A data dictionary can be built at this stage where the metadata is captured from the business perspective. Each attribute in business terms is defined and this enables easy exploration of data during data discovery.

This metadata is a means of cataloguing the existing data on the Data Lake to enable data discovery and track the lineage of data to know how data moves from the initial raw form across the data pipeline. Metadata capture will be discussed in detail in [Chapter 5](#), *Data Governance*.

Deep Integrity checks

As we have seen in [Chapter 1](#), *The Need for Data Lake*, the data in the Raw Zone is deeply validated. The following integrity checks can be performed on the data.

Bit Level Integrity checks

Data corruption might happen on the physical storage media such as the hard disks of the Hadoop cluster nodes. When the data is stored for an extended period of time, it tends to get corrupted; this is known as bit rot.

Blocks that comprise a file can be verified periodically to detect any corruption. The checksum of the block is compared with the original checksum and bad blocks can be identified. These bad blocks are identified and replaced as and when they are detected so that the file level integrity is maintained at all times.

Periodic checksum checks

A checksum can be used to detect the loss of data integrity. The checksum of the data in the Raw Storage Zone is calculated and stored as part of the

metadata. The checksum is verified on all subsequent reads to ensure that data integrity is maintained.

The checksum of the data can be periodically validated against the original checksum to detect data corruption.

There are multiple integrity assurance techniques, **Mirroring** is one such technique where two or more copies of data are maintained. If the loss of data integrity is detected, the corrupted data can be replaced with the copy of good data.

Security and governance

Data security and access groups requiring access to the data, which is at rest and in-flight by users, applications, and the network, can be identified and the appropriate policies can be defined on who can access what data. Sensitive data can be identified and masked in this zone to comply with the enterprise policies. Governance would be applied on the data by having the security policies in place to enable authenticated and authorized access to the data in both the Transient and Raw Zones. This is performed as a part of the Security and Governance layer and will be discussed in detail in [Chapter 5](#), *Data Governance*.

Information Lifecycle Management

Retention period is normally defined in the Raw Storage Zone; it states the use-by-date of the data. This is performed as a part of the Information Lifecycle Management layer. Generally, when data is stored for longer periods, its value decreases and the storage cost increases overtime. Defining retention periods and policies enables the Information lifecycle management layer to leverage tools that can archive, down-tier, or purge the data.

Practical Data Ingestion scenarios

Now that we have understood the Data Intake tier in detail, let us get a feel of how it is implemented in real-life situations when specifically dealing with Big Data. Here is a compilation of a few cases:

- A product-based company has huge structured data assets; their goal is to leverage data from external sources to improve their user experience. To address this, they have built a Big Data solution that ingests data from their structured data sources and social networks that have data about the preferences, needs, and opinions of their users. The data from social media is integrated and analyzed with the enterprise's structured data to understand user preferences and glean insights on their products based on the user's opinion. This helped them to improve their user experience and increased sales.
- A leading credit card issuing bank has implemented a Big Data solution that ingests huge volumes of batch data such as past activity, credit history, geographical information, spending habits, trends, and real-time data such as transactional data into the Data Lake. All of this data is analyzed using a fraud detection algorithm that analyses each transaction in real time, matches it with historical data, checks for patterns and sends out alerts if the algorithm identifies patterns that indicate any suspicious or fraudulent activity.